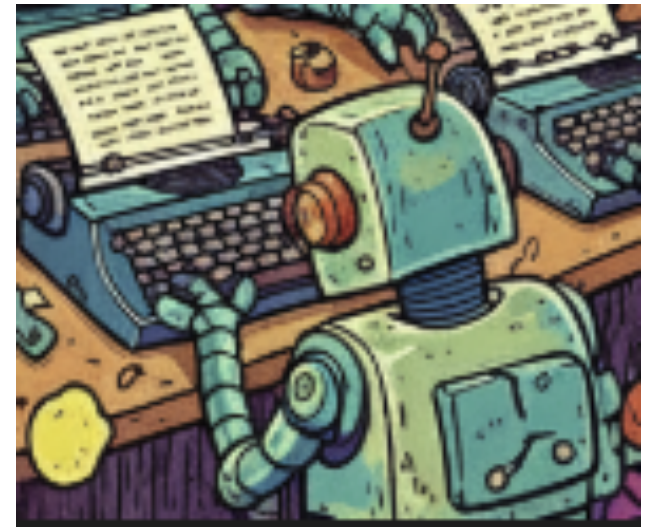




Agent Skills are a lightweight, portable format for packaging procedural knowledge and workflows. Structurally, a “skill” is a folder containing a mandatory SKILL.md file (which includes metadata and instructions) and optional subdirectories for scripts, reference materials, and templates.

The Skill Folder Structure:

- SKILL.md: (Required) Contains the name, description, and core instructions.
- scripts/: (Optional) Executable code for the agent.
- references/: (Optional) Supporting documentation.
- assets/: (Optional) Templates and other resources.

How They Work (Progressive Disclosure)

To maintain a small context footprint while allowing for vast capabilities, skills are loaded in three stages:

1. **Discovery:** The agent initially only reads the name and description of available skills to determine relevance.
2. **Activation:** Once a task matches a skill's description, the agent reads the full SKILL.md instructions.
3. **Execution:** The agent follows the instructions, executing scripts or referencing files as needed.

`qodo-get-rules` skill ensures the agent follows your team's specific coding conventions before it even writes a line of code.

- **Autohand Code CLI:** An ultra-fast, terminal-based coding agent. It uses a modular skill system to extend what it can do—meaning you can add new “reasoning patterns” or toolsets to the CLI without changing its core code.
- **Goose:** An open-source, extensible agent that goes beyond just code suggestions. It uses Agent Skills to allow the model to install, execute, and test code autonomously. It's designed to be a “universal” agent that you can plug into any workflow.

Summary of the Value

Because all these tools support the **Agent Skills** format, a skill you write for **Goose** will also work in **Mistral Vibe**, **Cursor**, or **Letta**. This prevents “vendor lock-in” and allows a community of developers to share specialized AI capabilities (like “How to fix a bug in Kubernetes”) that any agent can download and use. (<https://agentskills.io/home>)

Ecosystem and Support

The format was originally developed by Anthropic and is now an open standard. It is supported by various AI tools and clients, including:

- **IDEs & Editors:** VS Code, Cursor, GitHub Copilot, Roo Code, Trae.
- **Frameworks & Platforms:** Claude Code, Gemini CLI, Spring AI, Snowflake Cortex, Mistral AI Vibe, OpenHands.
- **Specialized Tools:** Factory, Letta, Qodo, Autohand Code CLI, Goose.

Website Navigation & Internal Links

The following internal domain links are available for deeper exploration:

- **Documentation Index:** <https://agentskills.io/llms.txt> (A machine-readable file for AI discovery).
- **Specification:** Details on the complete format and metadata requirements.
- **Quickstart:** A guide to creating your first skill.
- **Client Showcase:** A list of all compatible agentic clients.
- **For Skill Creators:**
 - Best Practices
 - Optimizing Descriptions
 - Evaluating Skills

- Using Scripts

For Client Implementors:

- Adding Skills Support

- **Community:** Links to the official Discord server and the

GitHub repository ([agentskills/agentskills](https://github.com/OpenHands/agentskills)).

The tools you mentioned are key players in the “agentic”

ecosystem, each integrating the **Agent Skills** standard to give AI

models specific “powers” or domain-specific knowledge.

Here is a breakdown of how these tools utilize the standard:

Frameworks & Platforms

These provide the underlying infrastructure to build and run

agents.

- **Spring AI:** A framework for Java developers that simplifies

building AI applications. It has a unified `skill()` method that

integrates with the Agent Skills API, allowing Java-based agents

to automatically handle code execution, file extraction, and

custom tool usage.

- **Snowflake Cortex:** A suite of AI features within the

Snowflake data platform. It uses “Cortex Agents” which can be

extended with skills to interact with structured data (SQL) and

unstructured sources, effectively giving the AI “tools” to process

data within your Snowflake environment.

- **Mistral AI Vibe:** Mistral’s open-source agent framework. It

natively follows the Agent Skills specification. You can create

skills in `~/vibe/skills/` with a `SKILL.md` file, and Vibe

will discover and load them based on your task. It also supports

“subagents” that use these skills for specialized work.

- **OpenHands (formerly OpenDevin):** A powerful open-source

framework for building autonomous agents that can write code

and operate a terminal. It uses Agent Skills to provide agents with

specialized “playbooks”—like knowing how to navigate a specific

complex codebase or use a particular API.

Specialized Agents & Coding CLIs

These are specific tools (often terminal-based) that you use

directly to get work done.

- **Factory:** Focused on “Droids” for autonomous engineering. It

uses skills to package “Expertise” (e.g., “how we implement React

components at this company”) and “Workflows” (e.g., “run this

specific deployment script”). It’s designed for enterprise

monorepos.

- **Letta:** Built to give agents long-term memory and specialized

capabilities. It implements the Agent Skills standard so you can

“install” skills simply by asking the agent to. For example, you

can give Letta a URL to a GitHub skill, and it will learn how to

use those new tools (like a PDF parser or a Slack GIF creator)

instantly.

- **Qodo (formerly CodiumAI):** Focuses on quality-first AI

coding. Qodo Skills “shift quality left” by injecting engineering

rules and standards directly into the agent. For example, their